

OmniRAG Studio

MULTI-SOURCE RAG ENGINE WITH REAL-TIME ANALYTICS

TOTAL DURATION	PRIMARY TOOLING	TARGET GOAL
6 Weeks (Step-by-Step)	Cursor Pro + FastAPI + Streamlit	Client-Ready Monetization

MASTER BLUEPRINT & ROADMAP OVERVIEW

OmniRAG Studio is an enterprise-grade AI chatbot system that ingests PDFs, CSVs, and Web URLs, provides precise citation-backed answers, and logs query analytics on an admin dashboard. Designed for high performance, modularity, and client acquisition.

6-Week Timeline Summary

WEEK	PHASE NAME	CORE FOCUS	DELIVERABLE
Week 1	Phase 1: Architecture & Setup	Cursor Pro setup, FastAPI structure, .env, and API integrations.	Base API Running
Week 2	Phase 2: Data Ingestion Pipeline	PDF, CSV, and Web scrapers with chunking & splitting strategy.	Ingestion Engine
Week 3	Phase 3: Vector DB & Retrieval	ChromaDB / Pinecone indexing, embedding generation & search.	Vector Retriever
Week 4	Phase 4: RAG Pipeline & Streaming	LangChain/LlamaIndex QA chain, prompt engineering & source citations.	Conversational RAG
Week 5	Phase 5: Analytics & Logging Engine	SQLite/PostgreSQL logger, latency tracking, token cost & metrics.	Analytics Backend
Week 6	Phase 6: Admin Dashboard & Launch	Streamlit/Next.js UI, Live Deployment, Loom video & GitHub setup.	Live Monetiable Product

Detailed Breakdown by Phase

WEEK 1

Phase 1: Foundation & Project Architecture Setup

Goal: Create a clean, modular repository structure and establish local API connectivity.

- **Environment Configuration:** Setup virtual environment (`venv`), `.env` file management for OpenAI, Anthropic, and Vector DB keys.
- **Modular Project Layout:** Create clean directory hierarchy (`/backend`, `/ingestion`, `/vectorstore`, `/analytics`, `/frontend`).
- **FastAPI Core Server:** Build base FastAPI server with CORS middleware, health check endpoint (`/health`), and custom error handlers.
- **Cursor Pro System Prompts:** Setup custom `.cursorrules` file to force clean PEP-8 python code and auto-documentation.

WEEK 2

Phase 2: Multi-Source Data Ingestion Engine

Goal: Parse and clean structured/unstructured data from PDFs, CSVs, and Web URLs.

- **PDF Parser:** Implement PyPDF / Unstructured document loaders with layout-aware text extraction.
- **CSV & Tabular Engine:** Build structured table reader converting CSV rows into semantically search-friendly chunks.
- **Web Crawler / Scraper:** Develop URL content extractor utilizing BeautifulSoup / LangChain WebBaseLoader with noise filtering.
- **Chunking Strategy:** Implement `RecursiveCharacterTextSplitter` with dynamic chunk size (e.g., 1000 tokens) and overlap (200 tokens).

WEEK 3

Phase 3: Embeddings & Vector Database Architecture

Goal: Convert text chunks into high-dimensional embeddings and store them for fast retrieval.

- **Embedding Integration:** Integrate OpenAI (`text-embedding-3-small`) or HuggingFace open-source embeddings.
- **Vector Store Setup:** Configure ChromaDB for rapid local development and Pinecone / Qdrant driver for cloud deployment.
- **Indexing & Persistence:** Build vector store indexing pipelines with metadata tracking (source, page number, row ID, URL).
- **Similarity Search:** Build retriever with Maximum Marginal Relevance (MMR) to prevent redundant context retrieval.

WEEK 4

Phase 4: RAG Pipeline, Prompting & Source Citations

Goal: Construct the conversational AI engine with strict anti-hallucination guardrails and citations.

- **Prompt Engineering:** Design system prompts strictly instructing the LLM to answer only using provided context.
- **LangChain / LlamaIndex Chains:** Construct conversational RAG chains with dynamic chat history memory management.
- **Source Attribution & Citations:** Format outputs to explicitly cite exact sources (e.g., `[Doc: Contract.pdf, Page 4]`).
- **Streaming Endpoint:** Implement Server-Sent Events (SSE) in FastAPI for real-time streaming response UI.

WEEK 5

Phase 5: Analytics Engine & Admin Logging

Goal: Track user interactions, monitor response accuracy, and calculate API operational costs.

- **Analytics Database:** Setup SQLite / PostgreSQL schema to store query history, timestamps, latency, and session IDs.
- **Metrics Engine:** Track token usage (Prompt vs Completion), estimated USD cost per query, and retrieved chunk scores.
- **Unresolved Query Tracker:** Automatically flag queries where the system responded with low confidence or fallback.
- **Analytics REST APIs:** Expose endpoint (`/api/v1/analytics/summary`) returning top questions, latency distributions, and total cost.

WEEK 6

Phase 6: Interactive UI, Deployment & Monetization Kit

Goal: Build the dual frontend (User Chat + Admin Dashboard), deploy live, and setup GitHub pitch kit.

- **Streamlit / Next.js Dual UI:** Build Tab 1 (User Chatbot + File Upload) and Tab 2 (Admin Analytics & Metrics Dashboard).
- **Cloud Deployment:** Deploy backend to Render/Railway and frontend to Streamlit Community Cloud / Vercel.
- **Portfolio Kit:** Record a 45-second Loom video demo showcasing data upload, citation answer, and live analytics update.
- **GitHub Repository Polish:** Draft a top-tier `README.md` with system design architecture diagrams, setup commands, and demo GIFs.

FINAL PRODUCTION PACKAGE DELIVERABLES

- Full Modular Source Code (Production-ready Python FastAPI + Streamlit backend/frontend).
- Live Working Interactive URL (Deployable for client demos on Upwork/LinkedIn).
- High-Impact Loom Video Script & Client Outreach Pitch Templates.